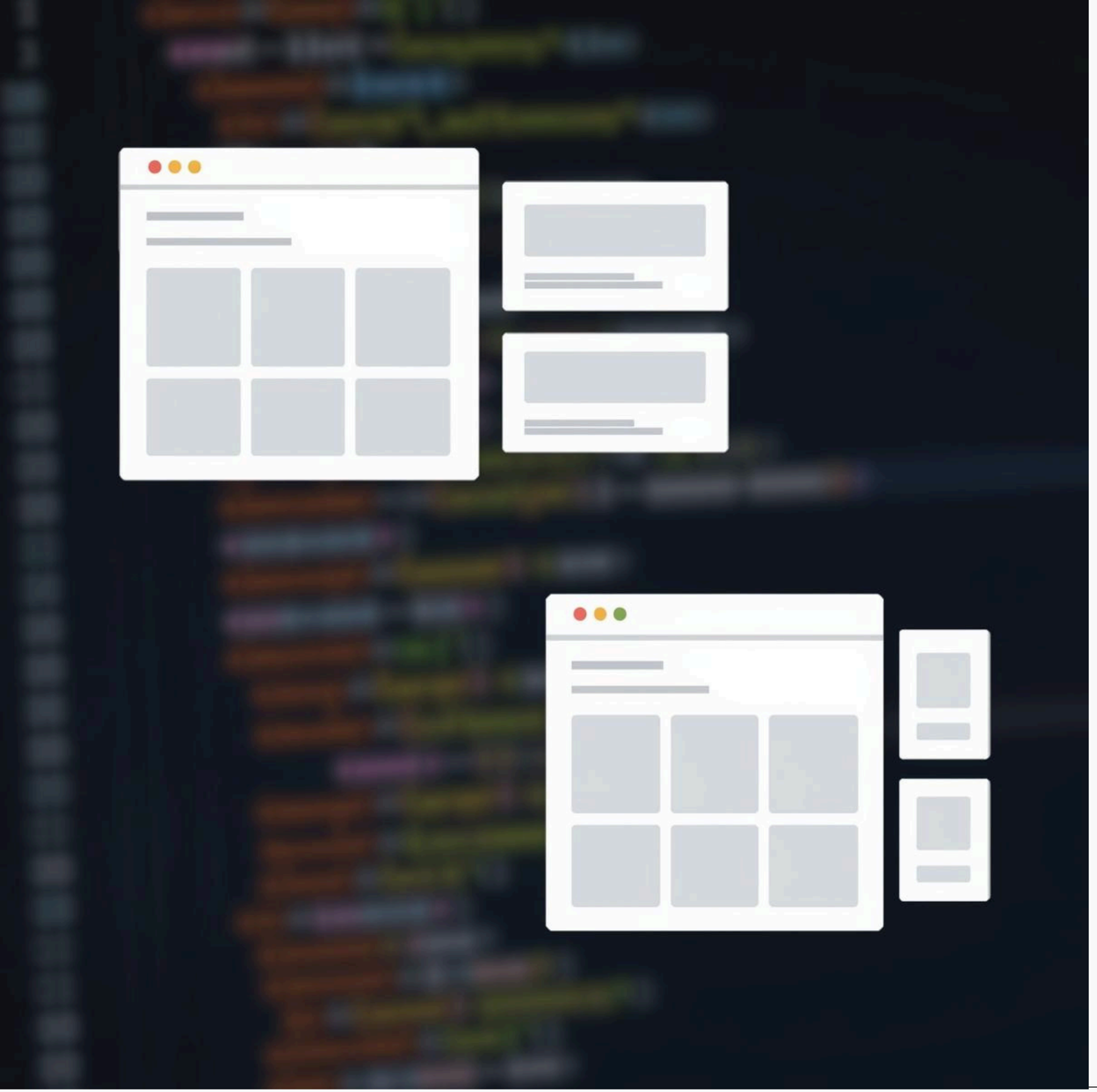


CSS

Layout

CREATED BY: ALEKSANDRA NOVÁK

Positioning

Positioning defines how an element is placed in the document and which element is used as a reference point for this placement. Placement is controled by top/left/right/bottom properties. Check out MDN Position for interactive presentation.

There are 5 position values:

- **static**
  - the default value for all elements, which follows the normal document flow - elements appear exactly where they are placed in the markup
  - not affected by top/left/right/bottom even if added

- **relative**
  - the element stays in normal flow and keeps its reserved space. However, when offseet is applied only its visual box moves
  - top/left/right/bottom shift the element relative to its original position - other elements behave as if it never moved
  - z-index works on relative elements, so they can be layered above/below others

- **fixed**
  - element is placed in relation to the viewport, so it stays in the same place even while scrolling the page
  - top/left/right/bottom are always calculated relative to the viewport box
  - element is remoed from the normal flow, so it may overlap other content - z-index can be used to control layout

- **sticky**
  - behaves like relative until the element reaches a boundary (top/left/right/bottom), and then it “sticks” to that boundary and stays there
  - the offsets apply relative to the element’s scrolling container, not the viewport itself
  - it stays in the normal document flow, but can still overlap content when stuck, so z-index works normally if layering needs adjustment

- **absolute**
  - the element leaves normal flow and no longer takes up space
  - top / left / right / bottom are calculated relative to the first non-static ancestor (relative, absolute, fixed, or sticky). If no such ancestor exists, it anchors to the initial containing block (visually, it would correspond typically to viewport), which may lead to several consequences, e.g element may appear as position correctly, however changing boundary, border, resizing viewport etc. may expose that it is not actually relative to what we intended.
  - z-index works normally for layering
  - quite often used with position: relative

Read more about **z-index** on MDN

REMARK: The default for top/right/bottom/left is auto. It means that browser will try to make some default positioning.

Float

Float pushes an element to the **left** or **right** and lets surrounding text or inline elements wrap around it. We apply float directly to the element that should shift (often an image). It can seem a bit unnatural as for example, quite often there is text that flows around an image, whereas from CSS point of view floated image determines text layout.

If we want later content to stop wrapping, we use **clear** on the following elements.

Float used to be a layout tool, but today it is avoided for full layouts as better methods were created (e.g. flexbox/grid).



A Complete Guide to CSS Flexbox

Our comprehensive guide to CSS flexbox layout. This complete guide explains everything about flexbox, focusing on all the different possible properties for the parent element (the flex container) and the child elements (the flex items). It also…

CSS-Tricks / Oct 2

FLEXBOX GUIDE

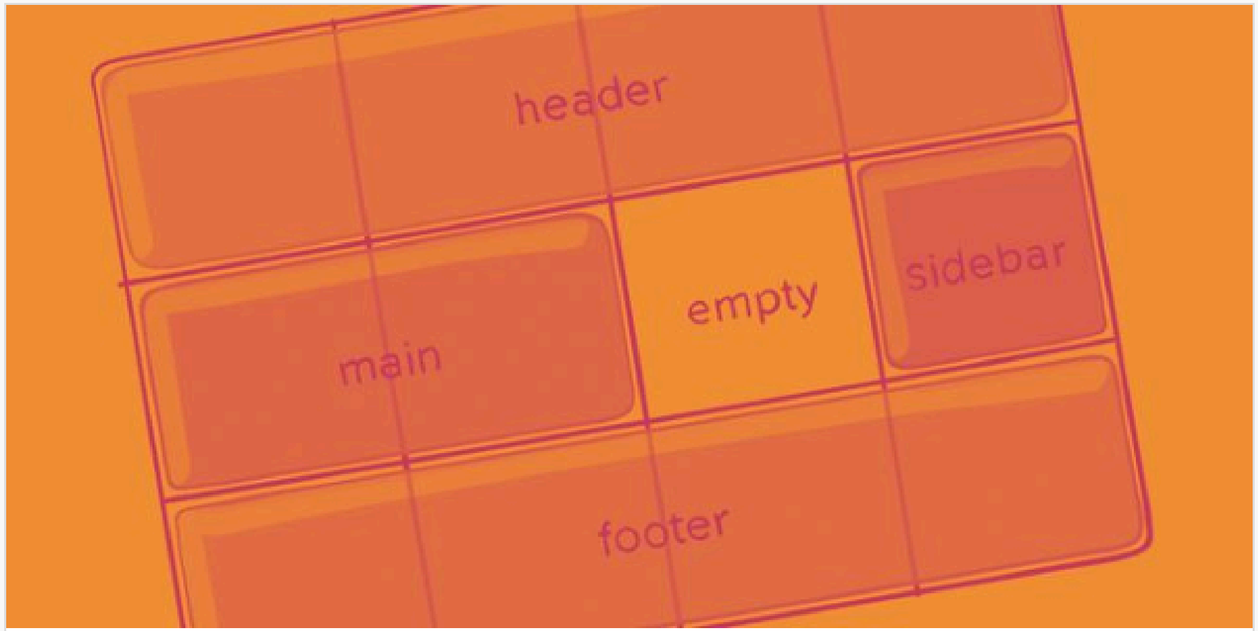


Flexbox Froggy

A game for learning CSS flexbox

flexboxfroggy.com

FLEXBOX GAME!!!!

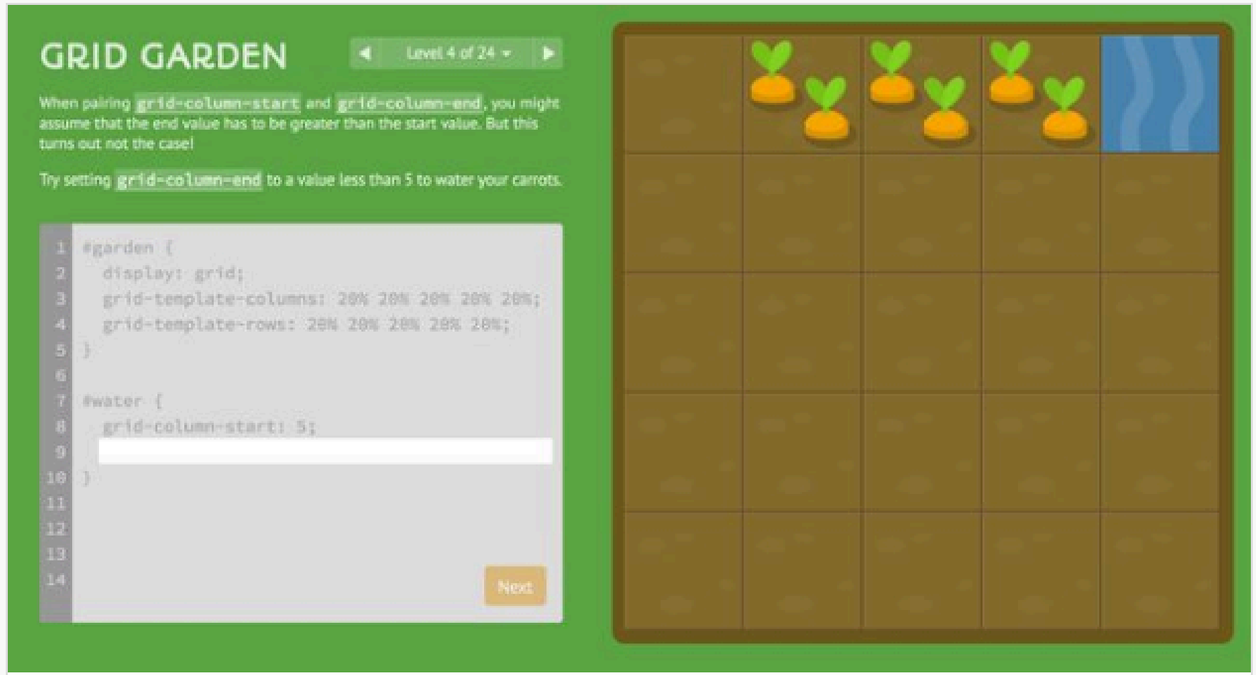


A Complete Guide to CSS Grid Layout

Our comprehensive guide to CSS grid, focusing on all the settings both for the grid parent container and the grid child elements.

CSS-Tricks / Sep 30

GRID GUIDE



Grid Garden

A game for learning CSS grid layout

cssgridgarden.com

GRID GAME!!!!

Responsive Web Design (RWD)

Approach to building pages that adapt to different screen sizes making webpage content readable on any device.

The most important ones are:

- **Responsive/Relative units (%, rem, vw/vh)**: layouts scale naturally with available space
- **Media queries** (**@media**): change layout/styling at specific breakpoints (e.g. when width of the screen is 600px or below)
- **Flexbox:** best for one-dimensional layouts as it aligns items in a single row or a single column. Great for nav bars, button groups, centering, distributing space between items etc. **FLEXBOX GUIDE**
- **Grid:** best for for two-dimensional layouts as it controls both rows and columns at the same time. Grat for full page structures, cards, galleries, forms, dashboards. **GRID GUIDE**
- **Responsive properties**: using **clamp()** and **calc()**
- **Mobile-first**: starting with the smallest layout

Animations

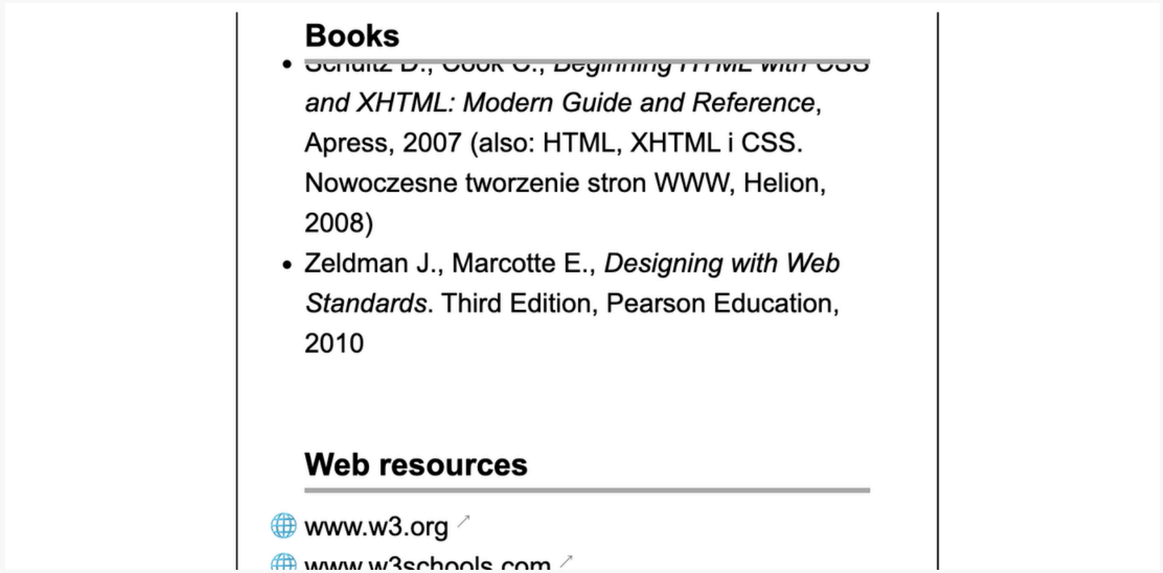
How to animate elements and effects:

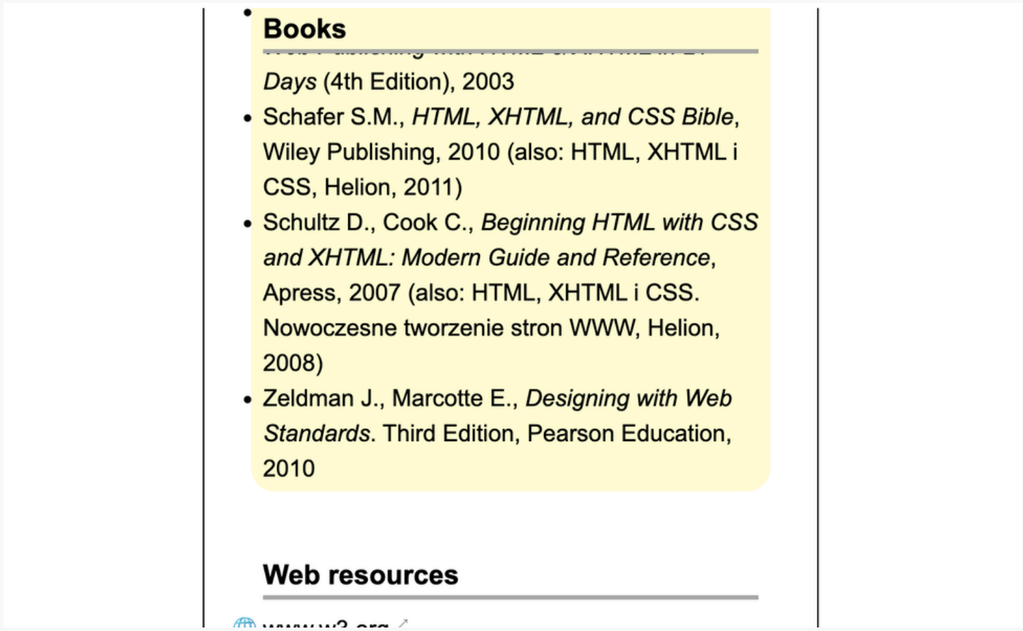
- **transform**: property that allows effects like scaling, rotating, and moving
- **transition**: property that reacts to change of state (CSS property values), define duration, smoothness (easing function) etc. It will be triggered by some pseudo-classes (like :hover) or JavaScript
- **@keyframes**: define stages of animation. Properties like color, opacity, transform etc. can be used inside.
- **animation**: property that “plays” set sequence of changes defined by @keyframes. It defines duration, repetition etc. User interaction is not needed

Modify website (index.html) from lab 3 following the tasks bellow:

TASK 1

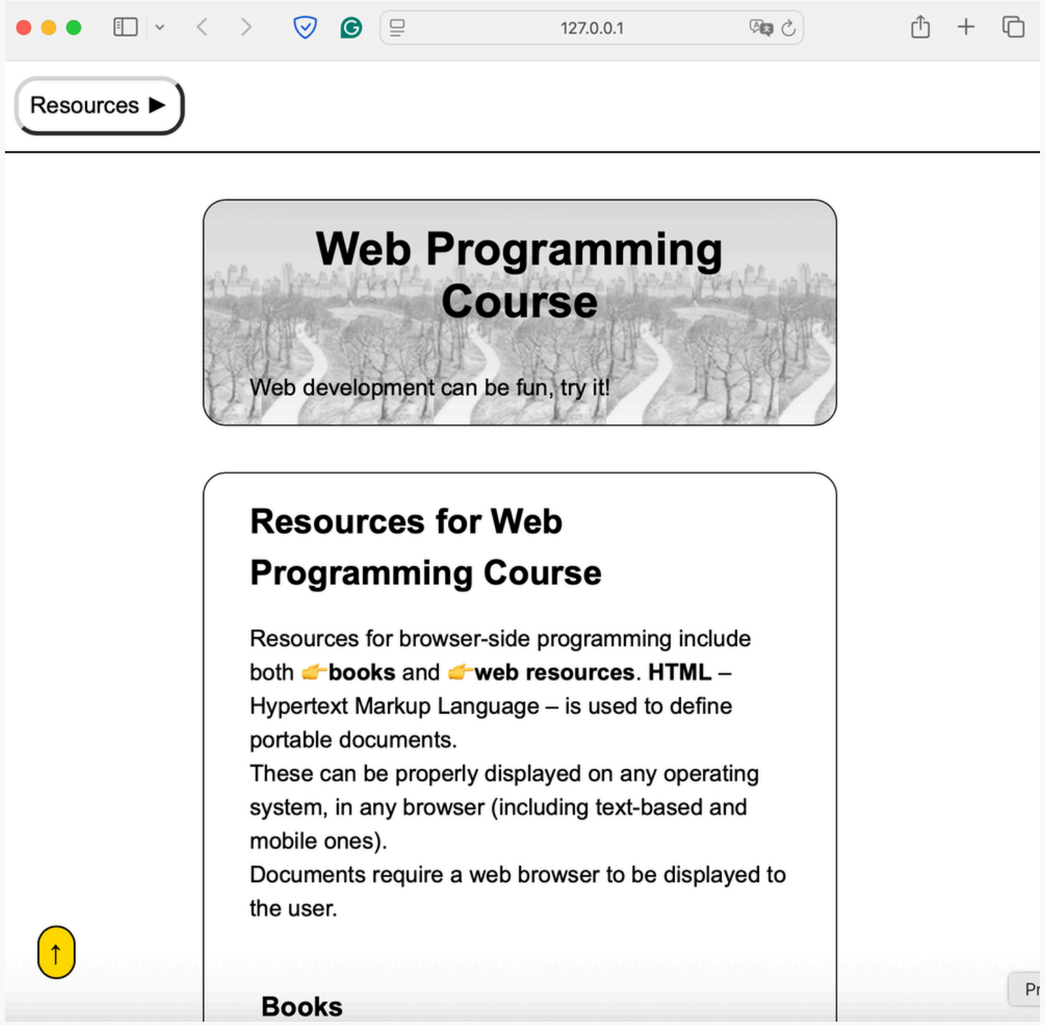
Make the “Books” and “Web resources” section headings stick to the top edge of the screen while scrolling, and stop sticking once their section has scrolled past. Remember to change the background color if section is selected (use **:target** pseudo-class). Use **position: sticky** for this effect.





TASK 2

Make the go-top link stay fixed in the bottom-left corner of the viewport, remaining visible even while scrolling.
Use **position: fixed**.

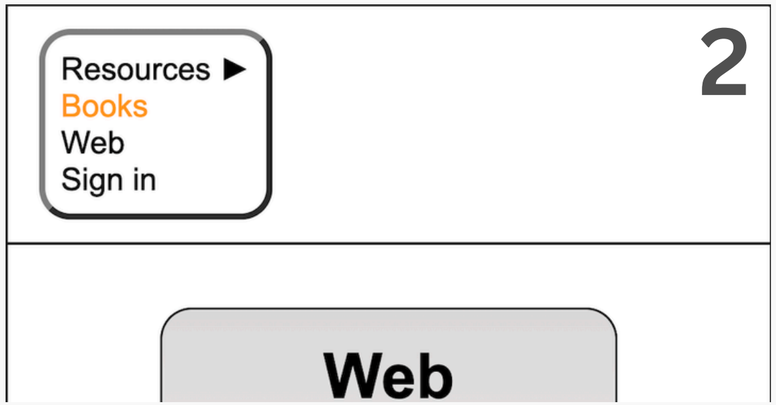


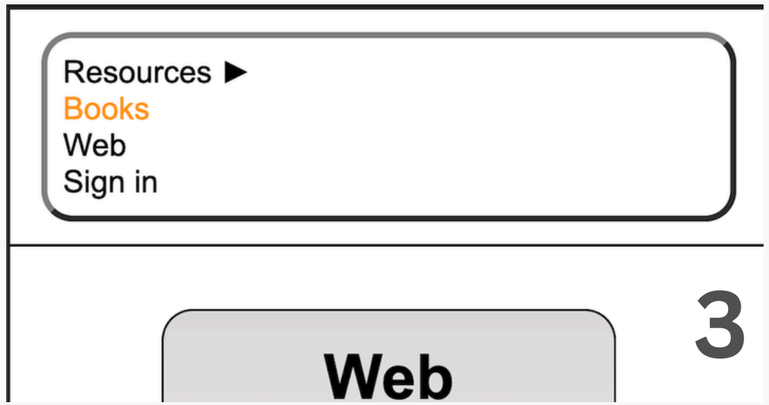
TASK 3

- Test out different menu configurations:
  - Achieve the result below using **display:block/display:inline-block** and using **display:flex, display:inline-flex, flex-direction:column**



  - Use **display:flex, display:inline-flex, flex-direction:column** on submenu/menu to get the result bellow.



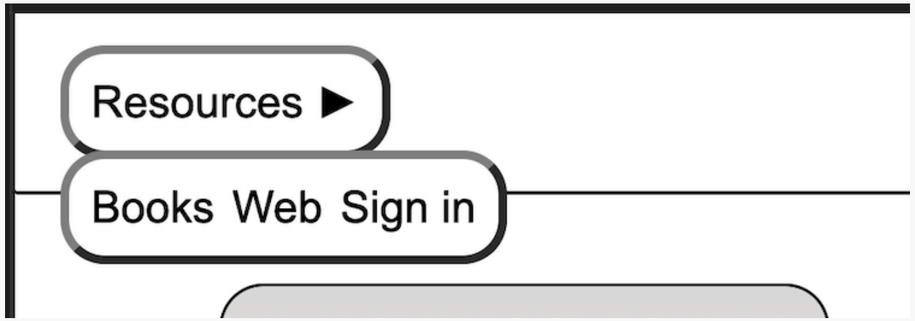




Change the navbar so that the submenu shown on hover is taken out of the normal flow and positioned relative to the menu label. Use **position: relative** on the menu and **position: absolute** on the submenu. The layout must not shift when the submenu appears, Be careful with boundaries (especially **left**). Submenu should open directly below the menu. Start from only adding border, positioning and see what happens with submenu and borders, then add boundaries. Try the following changes.

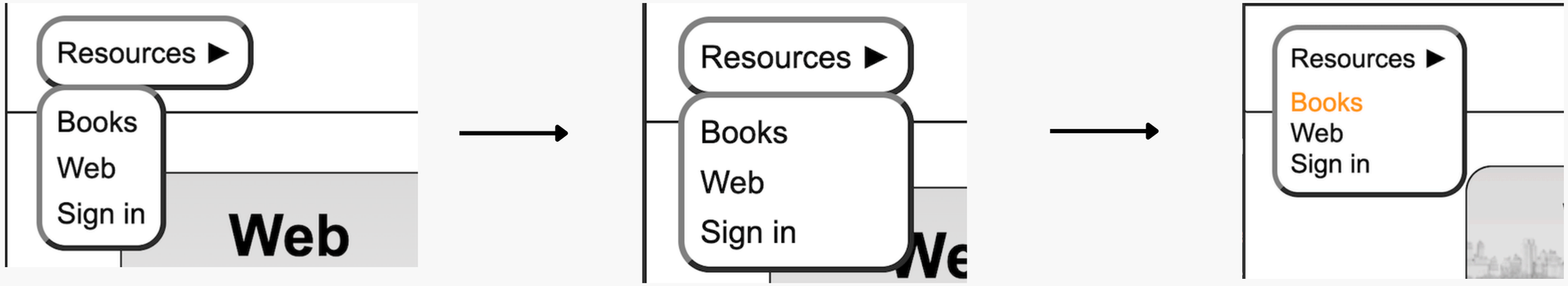
TASK 4

The menu's border should disappear on hover. Try adding ex. margin-top to .submenu:hover and see if it is possible to click on submenu



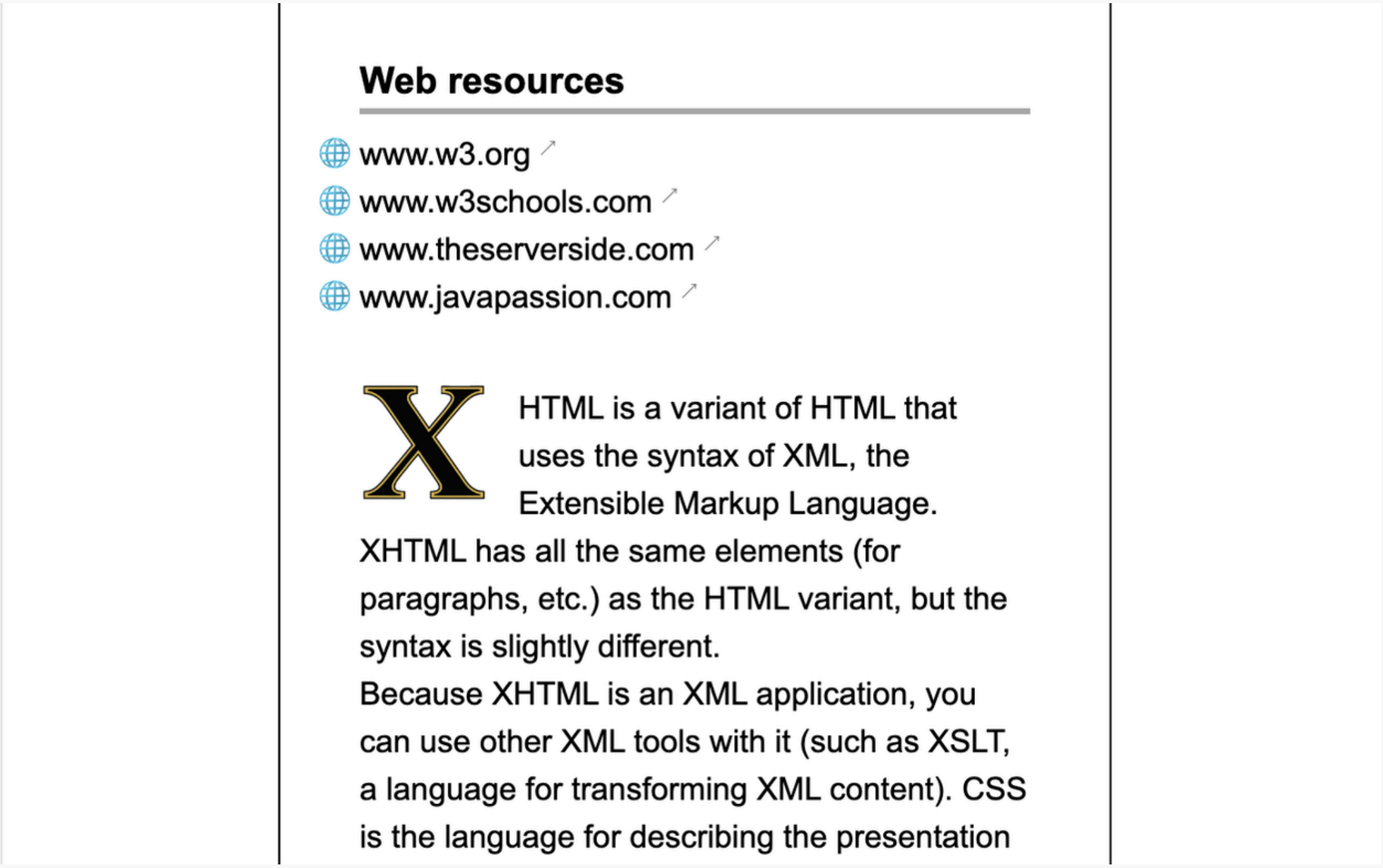
TASK 5

Change Display:flex, flex-direction:column on submenu (and menu) from task3 version 2. Try width:calc() to align menu and submenu boxes. Experiment with border (radius, border-top, border-bottom) to get the whole frame. Is the result accurate? Was it difficult to obtain sucha result?



TASK 6

- Add four paragraphs of text to the “Web resources” section and place an image inside the first paragraph. Use **float** on the image so the text wraps around it. As an experiment, apply the **clear** property to the following paragraphs to stop the float effect for the remaining text. Consider how the behavior would change if the image were placed outside the paragraph. Check the behaviour of image’s **margin, padding**

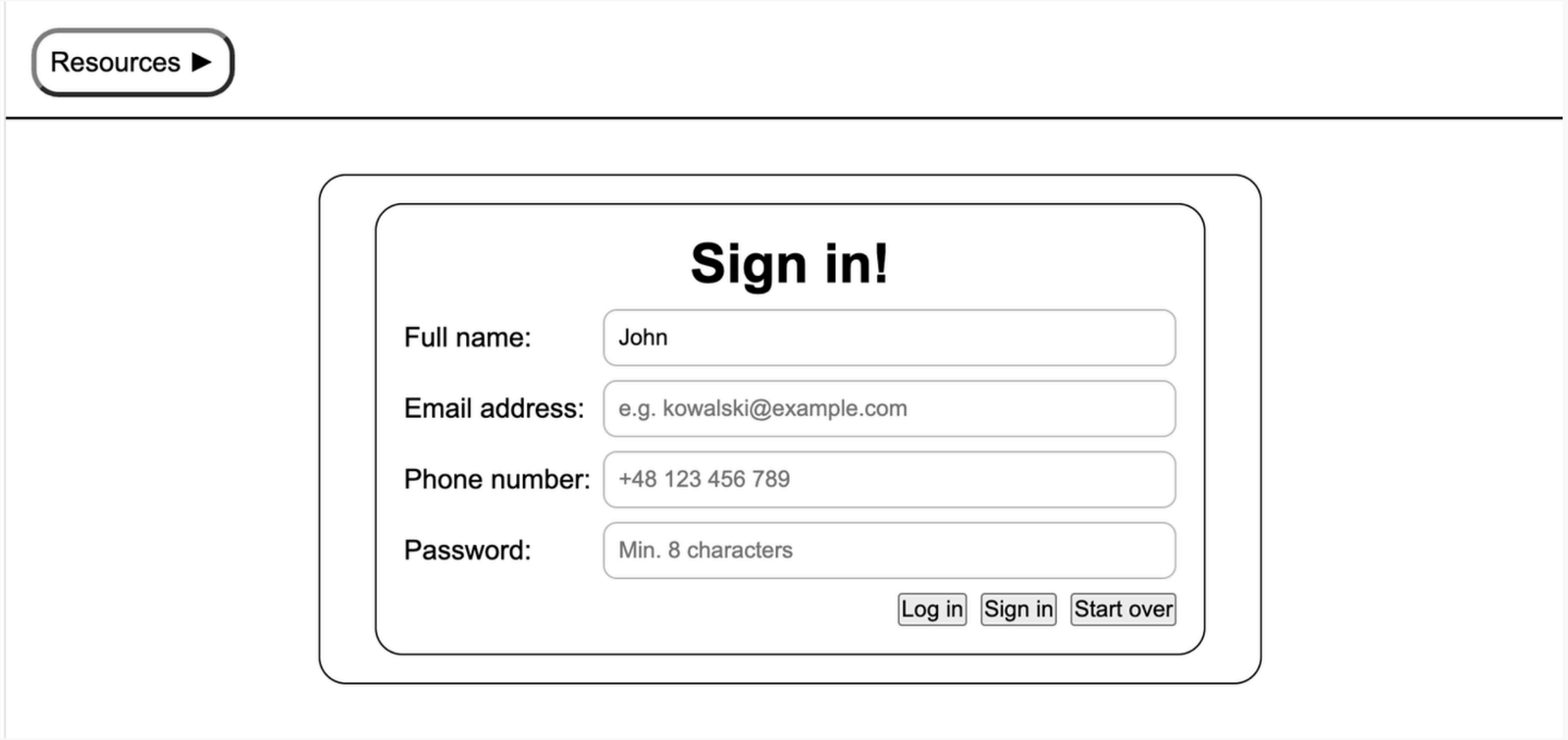


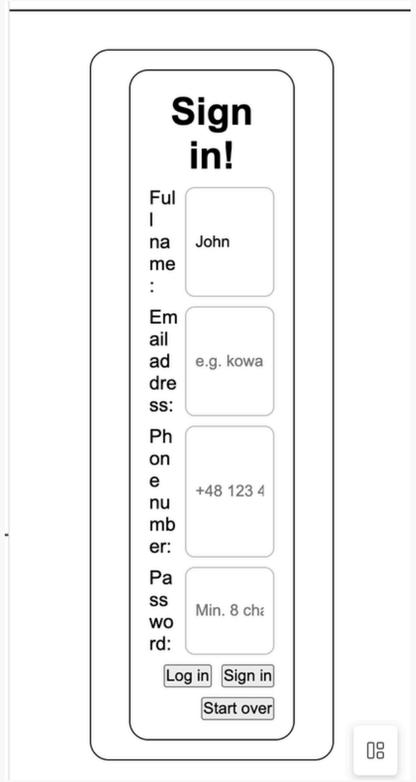
TASK 7

Add a new file form.html to your project.
Reuse the existing .container class from index.html. Place form inside it and center it at least vertically.

The form should be responsive and meet these requirements:

- “Sign in” is centered and spans the full row
- The label takes less space than the input, is aligned to the left, and vertically centered relative to its input
- Three buttons are aligned to the right

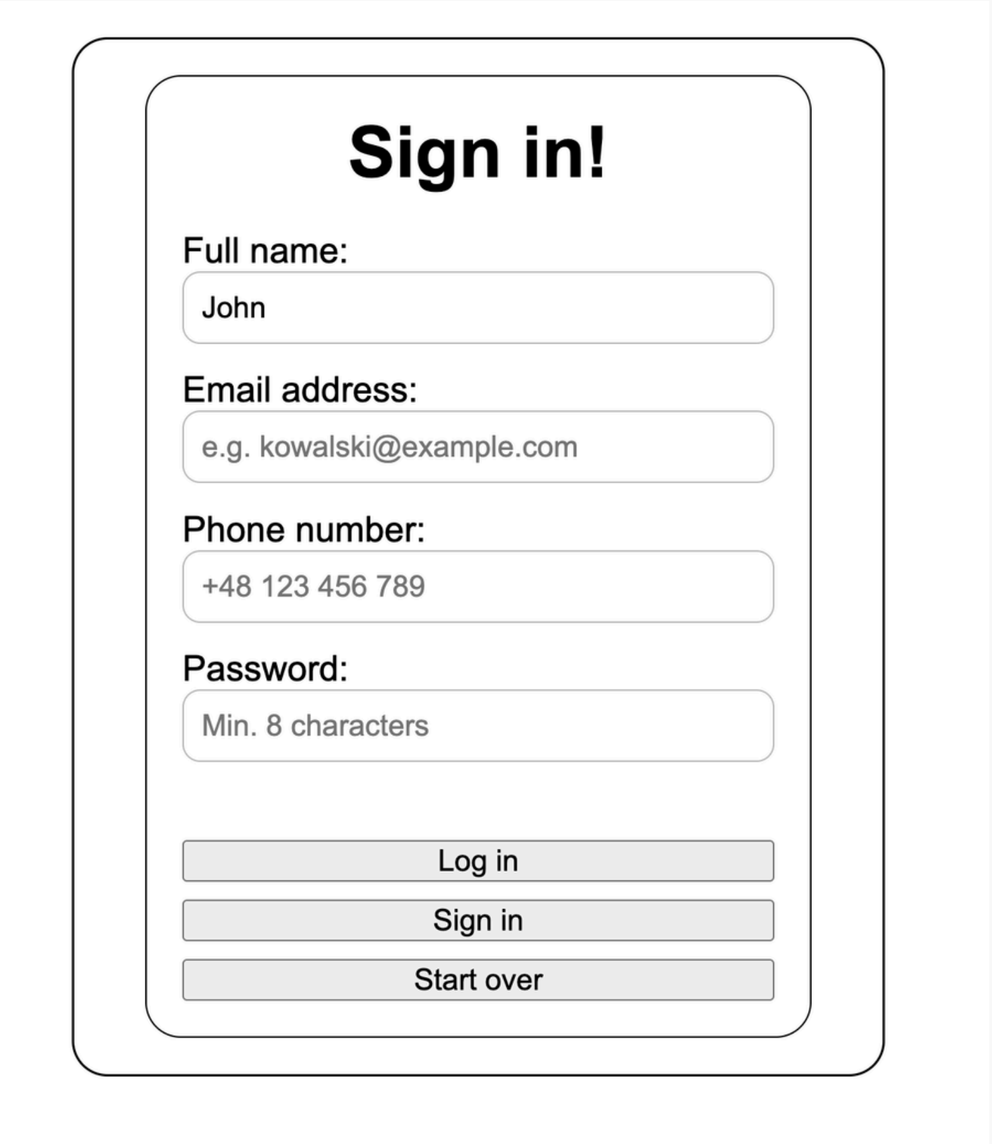




TASK 8

Use **@media** (media-query) to modify the form layout:

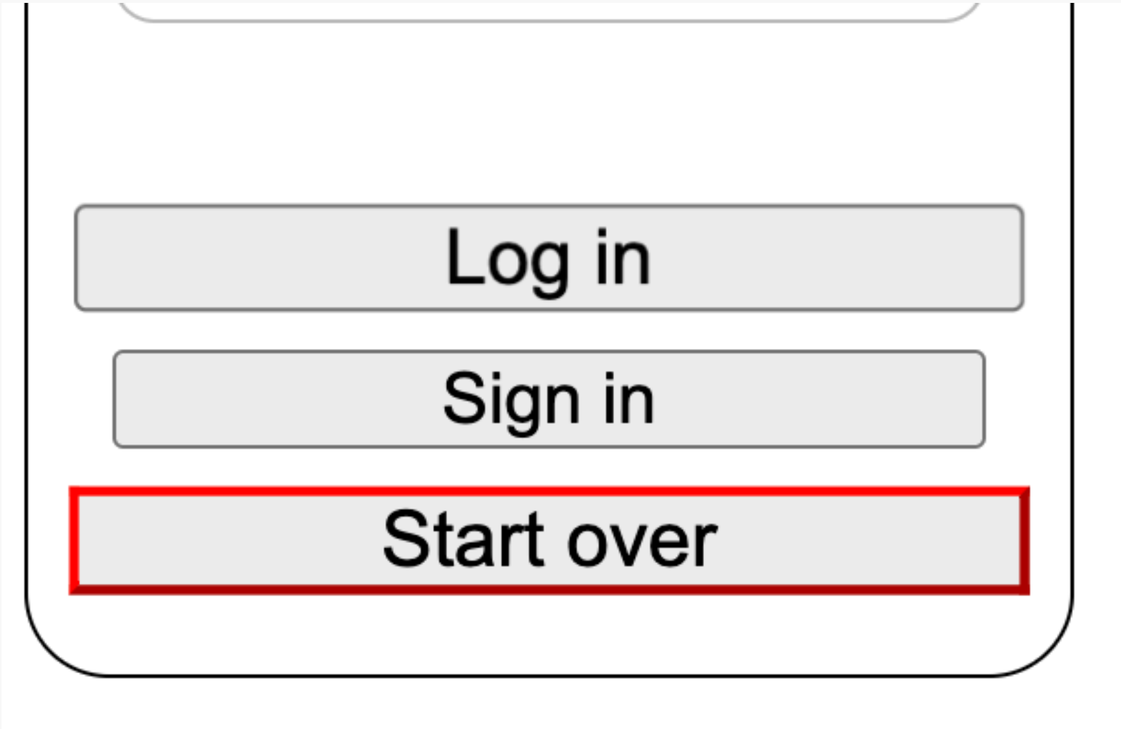
- Layout should change when width is below 650px
- Sign in! and buttons should be centered
- Labels should be above inputs



TASK 9

- Use **transform: scale(...)** to enlarge Start over button to 110% of its original size and change border color to red while **hovering** over it. Use **transition**:.... to set duration of these effects to ex. 0.3s

- Make Log in button pulse continuously using **@keyframes** and **animation** property (2 approaches):
  - Use from{} and to{} with **transform: scale(...)** in **@keyframes to scale button from 100% and 110%.** To make it “back-and-forth” make it repeat infinitely in alternating directions (using **animation**)
  - Use **0%, 50%, 100%** to get the same effect. change **animation** accordingly



Disclaimer

- Tasks were adapted from the course materials “CSS”, tasks 1-6, of the Web Application Programming/Programowanie Aplikacji WWW course by dr hab. inż. prof. uczelni Maciej Grzenda (PhD at the time of creation), website: http://www.mini.pw.edu.pl/~grzendam .

- The content presented in the presentation is consistent with the above-mentioned materials.

- The content is primarily based on the HTML Living Standard/CSS/MDN documentation referenced on each slide, unless stated otherwise.

- All materials are provided for educational and non-commercial use only.